



END SEMESTER ASSESSMENT (ESA) - Jan - May 2024

UE19CS351 - Compiler Design

Total Marks : 100.0

1.a. i) Define Compiler. Explain in brief how it is different from an Interpreter.
ii) Explain in brief (any two) types of compilers. (4.0 Marks)

1.b. What is Input buffering in lexical analysis? Explain in brief the commonly used buffering methods? (4.0 Marks)

1.c. Explain the design of a Lexical analyzer with a neat diagram. (4.0 Marks)

1.d. Explain in brief the Frontend and backend of a Compiler.

(4.0 Marks)

1.e. What is Lex and Yacc? Explain in brief.

(4.0 Marks)

2.a. Explain in detail Panic-Mode and Phrase-Level (or Statement Mode) Error-Recovery Strategies. (4.0 Marks)

2.b. For the following grammar answer the following:

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \lambda$

$B \rightarrow \lambda$

1. Construct first and follow table.
2. Construct LL(1) table and Mention whether the grammar is in LL(1) or not.
3. If the grammar is in LL(1), parse the string: ba

Note: λ is epsilon(i.e., null character)

(6.0 Marks)

2.c. Consider the following grammar:

$S \rightarrow AB \mid a$

$A \rightarrow a$

$B \rightarrow b$

1. Construct LR(0) automata
 2. Construct SLR parsing table to check whether the given grammar is SLR or not.
- (6.0 Marks)

2.d. With simple example explain the following terms (1 mark each):

- i) Left-recursive grammar
- ii) Leftmost derivation
- iii) Ambiguous grammar
- iv) LR(0) item

(4.0 Marks)

3.a. Explain the following

1. Synthesized Attributes
2. Inherited Attributes
3. L-attributed SDT and
4. S-attributed SDT

(8.0 Marks)

3.b. Consider the following grammar for simple arithmetic expressions:

$L \rightarrow En$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

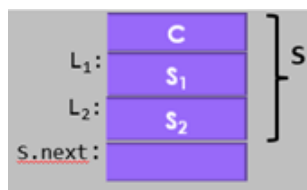
$F \rightarrow \text{digit}$

Write an SDD for arithmetic expression evaluation (i.e., desk calculator) during parsing.

(4.0 Marks)

3.c. Write syntax directed definition (SDD) to generate intermediate code for if-else statement.

Given: $S \rightarrow \text{if}(C) S_1 \text{ else } S_2$



(4.0 Marks)

3.d. Explain the following

- (i) Annotated Parse tree
- (ii) Dependency graph

(4.0 Marks)

4.a. Write short notes on:

- i) Triples
- ii) Static Single Assignment (SSA)

(5.0 Marks)

4.b. Construct control-flow graph (CFG) for the following three-address code.

(5.0 Marks)

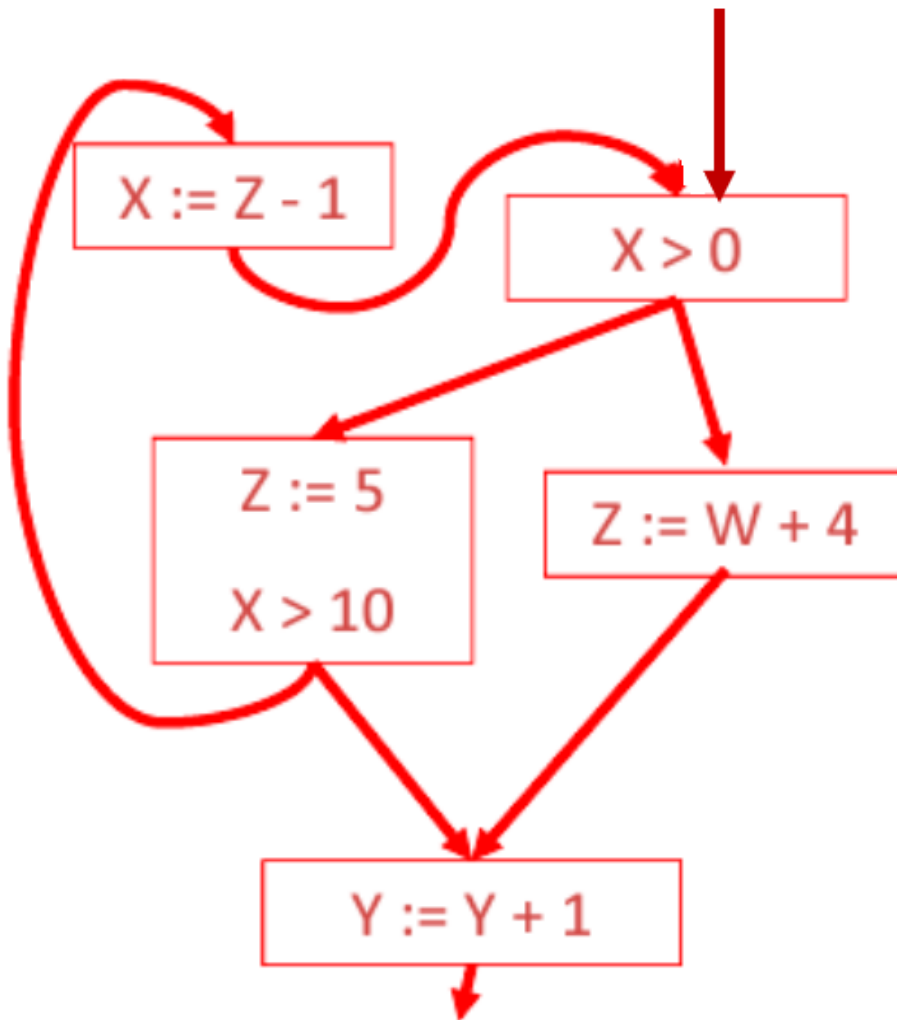
```
x = n
y = m
iffalse x < y goto L1
t1 = x + 1
x = t1
t2 = y - 1
y = t2
goto L2
L1:
  y = x + 2;
L2:
  z = x * y;
  return z;
```

4.c. What is code optimization? Explain with an example any two Optimization techniques.

(5.0 Marks)

4.d. Calculate use[B], def[B] and then find IN[B], OUT[B] by applying *Live-variable Analysis Algorithm* on the following flow graph.

(5.0 Marks)



5.a. Explain in brief any four issues in the design of a code generator. (4.0 Marks)

5.b. Explain the following

1. Activation record
2. Activation tree
3. Calling sequence

(6.0 Marks)

5.c. Explain in brief the different ways in which a procedure can access nonlocal data on a runtime Stack.

(4.0 Marks)

5.d. Generate target code sequence for the following basic block three address statements with register and address descriptors.

1. $x := y + z$
2. $z := x * x$
3. $y := z$
4. $x := y + z$

Note 1: Assuming only two registers (R1 and R2) are available and all the variables are live on exit from the basic block. No temporary variables are used.

Note 2: Assuming that all arithmetic operations take their operands from registers.
(6.0 Marks)